REPUBLIC OF THE PHILIPPINES
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
College of Computer and Information Sciences
PUP Sta. Mesa

BAGONG PILIPINAS

Principles of Programming Languages

Programming Language Proposal

COSC-303

Standard Measurement Language (SML): A Unit-Safe Programming Language for Physical Quantities

Submitted by:

Laman, Allen Marcel F.

Mallan, Randyl John Kenneth S.

Pascual, Gerry C.

Pauso, Lorenzo Cobey

Plotado, Jan Haji E.

Yukawa, Rei Emerson O.

BSCS 3-4

I. Introduction

Standard Measurement Language (SML) is a small, statically checked procedural programming language for calculations involving physical measurements. It is intended primarily for programmers performing introductory structural-engineering and scientific calculations, where confusing units can make a valid formula produce a meaningless result. Numerical values may include physical units, such as `100m` and `9.58s`, while values such as `0.5` remain valid dimensionless numbers.

SML checks whether the units used in an expression are compatible. An addition such as `10km + 5s` is invalid because length and time are different dimensions. Multiplication, division, and integer powers automatically derive the dimension of their result. Thus, dividing a distance by a time produces a speed, while multiplying mass by acceleration produces force.

```
let distance = 100m;
let time = 9.58s;
let speed = distance / time;

let mass = 50kg;
let acceleration = 9.8[m/s^2];
let force = mass * acceleration;
```

Conversions between compatible units are never implicit. The programmer must use the `as` operator to state the intended representation:

```
let total = 5km + (3000m as km);
```

SML supports primitive, dimensional, and exact-unit annotations, as well as local initializer inference. Its familiar syntax uses semicolons, brace-delimited blocks, and deliberately bounded unit expressions.

The physical model supports the base dimensions Length, Mass, and Time, together with the named derived dimension Force. Its initial physical units include `m`, `cm`, `mm`, `km`, `mi`, `ft`, `in`, `kg`, `s`, `min`, `N`, and `kN`. Volume remains a derived dimension written with an exact compound-unit annotation such as `[m^3]` rather than a separate named dimension. Currency, affine temperature, user-defined units, and automatic code generation are outside the scope of this version.

Inspiration for the Language

The inspiration is a real failure. In 1999, NASA lost the Mars Climate Orbiter, a mission of roughly 125 million USD, because one team's software produced thruster impulse data in imperial units (pound-force seconds) while another team's software expected metric units (Newton seconds). Both programs were "correct" on their own. Both used plain numbers. Nothing in the language could notice that the two numbers meant different things.

General-purpose languages usually represent measurements as ordinary integers or floating-point values. Units remain in comments, names such as `distance_km`, or the programmer's memory. Such languages can detect division by zero, but ordinarily cannot recognize incompatible measurements or unit conventions.

SML uses familiar C-like elements, including braces, semicolons, expressions, declarations, and procedural control flow. Its distinguishing feature is that units participate directly in quantities, expressions, declarations, and conversions.

Target Users

1. **Students in physics and engineering courses** who write programs for lab computations. A unit mistake that would cost marks on paper is caught by the compiler instead.
2. **Engineers prototyping calculations** (construction, physics) who want a safety net before moving to a larger language.

SML is not meant for building large software systems. Like a calculator that refuses to give nonsense answers, it is a focused tool for computations where units matter.

The New Principle

SML's uniqueness principle is *Meaning Before Math*: a numerical operation is accepted only when the meanings represented by its operands are compatible. The principle is expressed through three connected design rules:

Design Feature	What it does	Why it is an improvement
Units in quantity syntax	Units are written directly with values, as in `100m` and `9.8[m/s^2]`.	Measurement meaning is recorded in the program instead of only in comments or variable names
Automatic, bounded unit algebra	SML derives result dimensions from operations: `m / s` produces `m/s`, while `m * m` produces `m^2`.	Derived units are computed automatically from expressions, while explicit compound-unit annotations remain bounded and checkable.
No Silent Conversion rule	Two values of the same dimension but different units (`km` and `mi`) cannot be added or compared until one is converted with as.	Addresses the same class of unit-interface mistake like the Mars Orbiter
Unit contracts on functions	Function signatures state their dimensions (`fn speed(distance: Length, time: Time): [m/s]`). The compiler checks both the arguments and the returned value.	Mismatches are found at the boundary between teams or modules, which is where the Orbiter failed.
Unit-aware output	`print` writes the unit together with the number (`10.44 m/s`).	Results are self-explaining, so a wrong unit is visible on screen.

SML is designed to accept an operation only when its operands satisfy the operation's type, dimension, and unit-representation rules. It rejects incompatible operations during static checking, before they can produce plausible-looking numerical results.

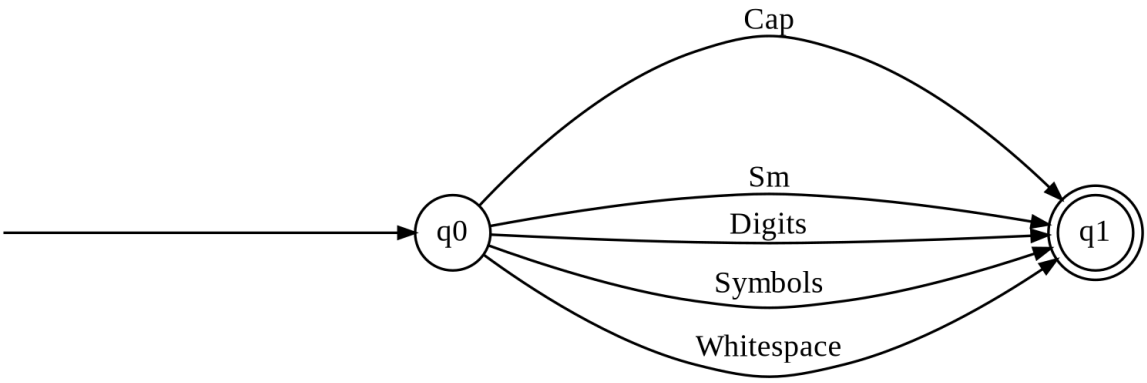
II. Syntatic Elements of Language

1. Character Set

- Cap = { A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z }
- Sm = { a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z }
- Alpha = { Cap, Sm }
- Digits = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- Symbols = { ., +, -, *, /, %, ^, <, >, =, !, &, |, (,), {, }, [,], ;, :, ,, ", _ }
- Whitespace = { <space>, <horizontal-tab>, <carriage-return>, <line-feed> }

- Character = { Alpha, Digits, Symbols, Whitespace }

Machine for Character Set:



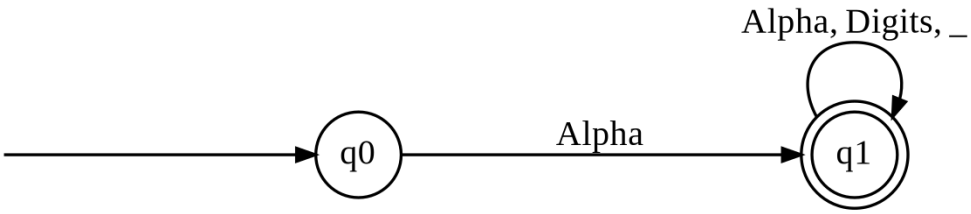
2. Identifiers

- Identifier = Alpha (Alpha | Digits | _)*

Rules:

- An identifier must begin with an uppercase or lowercase ASCII letter.
- Later characters may be ASCII letters, digits, or underscores.
- An identifier cannot begin with an underscore or digit.
- Consecutive and trailing underscores are allowed.
- Identifiers are case-sensitive. For example, speed, Speed, and SPEED are different names.
- Keywords, reserved words, primitive types, named dimensions, and noise words cannot be used as identifiers.
- A registered unit symbol, such as `m`, `ft`, `in`, `mi`, `s`, or `min`, cannot be redeclared as a value identifier.

Machine for Identifier:



- Regular Expression: [A-Za-z][A-Za-z0-9_]*

3. Literals

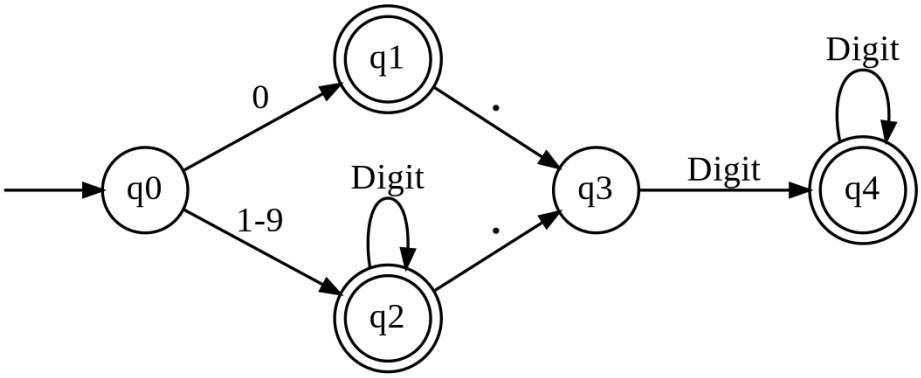
A literal is a source form that directly represents a value. SML supports numeric, string, Boolean, and quantity literals. A leading + or - is an expression operator and is not part of a literal.

3.1. Numeric Literals

Numeric literals contain decimal digits and may include one decimal point. An integer part is either the single digit 0 or a nonzero digit followed by zero or more digits. Therefore, leading zeros are not permitted. SML also does not support leading-dot forms, trailing-dot forms, exponent notation, digit separators, or alternate number bases.

- NonZeroDigit = { 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- Dot = { . }
- IntegerLiteral = 0 | NonZeroDigit Digit*
- DecimalLiteral = IntegerLiteral Dot Digit Digit*
- NumberLiteral = IntegerLiteral | DecimalLiteral

Machine for Numeric Literals:



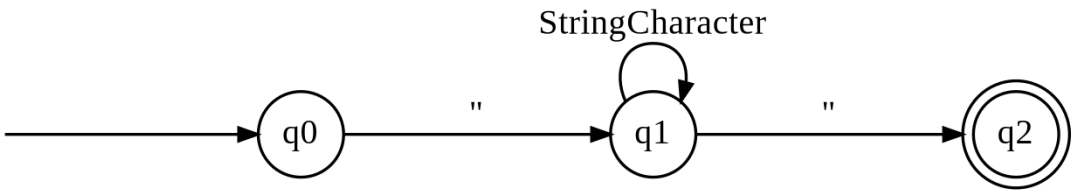
- Regular Expression: (0|[1-9][0-9]*)(\.[0-9]+)?

3.2. String and Boolean Literals

A string literal is enclosed in double quotation marks. It may contain canonical SML characters other than a double quotation mark, carriage return, or line feed. Empty strings are allowed. Escape sequences, embedded double quotation marks, and multiline strings are not supported in the current proposal.

- DoubleQuote = { " }
- StringCharacter = Character - { DoubleQuote, <carriage-return>, <line-feed> }
- StringLiteral = DoubleQuote StringCharacter* DoubleQuote

Machine for String Literals:



Boolean literals have exactly two case-sensitive spellings:

- BooleanLiteral = { true, false }

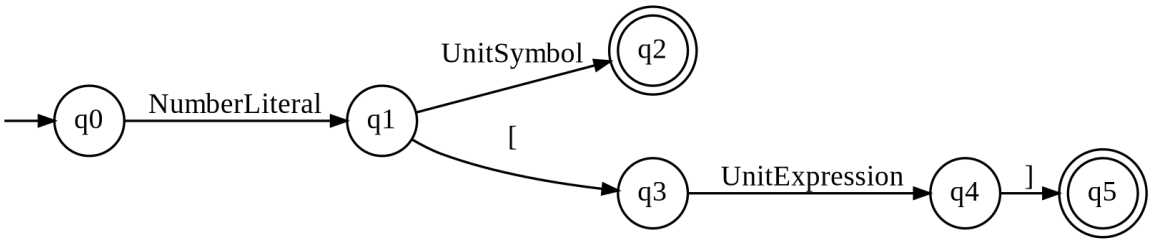
3.3. Quantity Literals

A quantity literal attaches a numeric magnitude to unit syntax. A simple quantity uses one unit symbol, while a compound quantity encloses a unit expression in square brackets.

- QuantityLiteral = SimpleQuantityLiteral | CompoundQuantityLiteral
- SimpleQuantityLiteral = NumberLiteral UnitSymbol
- CompoundQuantityLiteral = NumberLiteral "[" UnitExpression "]"
- UnitSymbol = Identifier

Compound quantities are written with square brackets like `9.8[m/s^2]`.

Machine for Quantity Literals:



Unit symbols share the lexical form of identifiers. The lexer does not decide whether an identifier is a registered unit; unit resolution occurs after parsing. The complete grammar and hard limits for UnitExpression are presented later under **Expressions**.

3.4. Built-in Units and Conversion Relationships

These relationships define the conversions supported by SML. They do not cause automatic conversion. When compatible quantities use different unit representations, the programmer must request the conversion explicitly with `as`.

Dimension	Symbol	Unit	Defined relationship
Length	`m`	meter	Canonical Length unit
Length	`cm`	centimeter	`1cm = 0.01m`
Length	`mm`	millimeter	`1mm = 0.001m`
Length	`km`	kilometer	`1km = 1000m`
Length	`in`	inch	`1in = 0.0254m`
Length	`ft`	foot	`1ft = 0.3048m`
Length	`mi`	mile	`1mi = 1609.344m`
Mass	`kg`	kilogram	Canonical Mass unit
Time	`s`	second	Canonical Time unit
Time	`min`	minute	`1min = 60s`
Force	`N`	newton	`1N = 1[kg*m/s^2]`
Force	`kN`	kilonewton	`1kN = 1000N`

For example:

```
let distance = 1km as m;  // 1000m
let force = 2kN as N;     // 2000N
```

Every unit symbol is recognized as one complete, case-sensitive spelling. For example, `km` is one unit symbol rather than `k` multiplied by `m`.

4. Operation Symbols

- AssignmentOperator = "=" | "+=" | "-=" | "*=" | "/="
- ArithmeticOperator = "+" | "-" | "*" | "/" | "%" | "^"
- UnaryOperator = "!" | "+" | "-"
- LogicalOperator = "!" | "&&" | "||"
- RelationalOperator = "==" | "!=" | "<=" | ">=" | "<" | ">"
- UpdateOperator = "++" | "--"

4.1. Assignment Operators

Operator	Example	Description
`=`	`length = 5m`	Stores the right-hand value in the named variable.
`+=`	`length += 1m`	Adds a value with the same exact unit representation and stores the result.
`-=`	`length -= 1m`	Subtracts a value with the same exact unit representation and stores the result.
`*=`	`length *= 2`	Multiplies by a dimensionless value and stores the result.
`/=`	`length /= 2`	Divides by a dimensionless value and stores the result.

- Regular Expression: (\+=|-=|*=|/=|=)

4.2. Arithmetic Operators

Operator	Example	Description
+	`5m + 2m`	Adds operands with the same exact unit representation.
-	`5m - 2m`	Subtracts operands with the same exact unit representation.

*	`3m * 2m`	Multiplies magnitudes and derives the product unit.
/	`100m / 9.5s`	Divides magnitudes and derives the quotient unit.
%	`5m % 2m`	Produces the remainder for operands with the same exact unit representation.
^	`3m ^ 2`	Raises a value and its unit to a power.

The exponent of `^` must be dimensionless. A dimensioned base requires a compile-time integer exponent, while a dimensionless base may use an integer or decimal exponent. Decimal exponents are never permitted inside a written UnitExpression.

- Regular Expression: (\+|-|*|/|%|\^)

4.3. Unary Operators

Operator	Example	Description
unary +	`+length`	Preserves the sign of a numeric value.
unary –	`-length`	Negates a numeric value.
!	`!valid`	Produces the logical opposite of a Boolean expression.
&&	`ready && safe`	Produces true only when both operands are true.
||	`ready || override`	Produces true when at least one operand is true.

- Regular Expression: (!|\+|-)

4.4. Logical Operators

Operator	Example	Description
!	`!valid`	Produces the logical opposite of a Boolean expression.
&&	`ready && safe`	Produces true only when both operands are true.
||	`ready || override`	Produces true when at least one operand is true.

- Regular Expression: (!|&&|\|\|)

4.5. Relational Operators

Operator	Meaning	Example
<	less than	`3m < 5m`
<=	less than or equal to	`3m <= 5m`
>	greater than	`5m > 3m`
>=	greater than or equal to	`5m >= 3m`
==	equal to	`5m == 5m`
!=	not equal to	`5m != 3m`

Both operands must have the same unit. `5m < 3s` and `5km < 3mi` are compile-time errors.

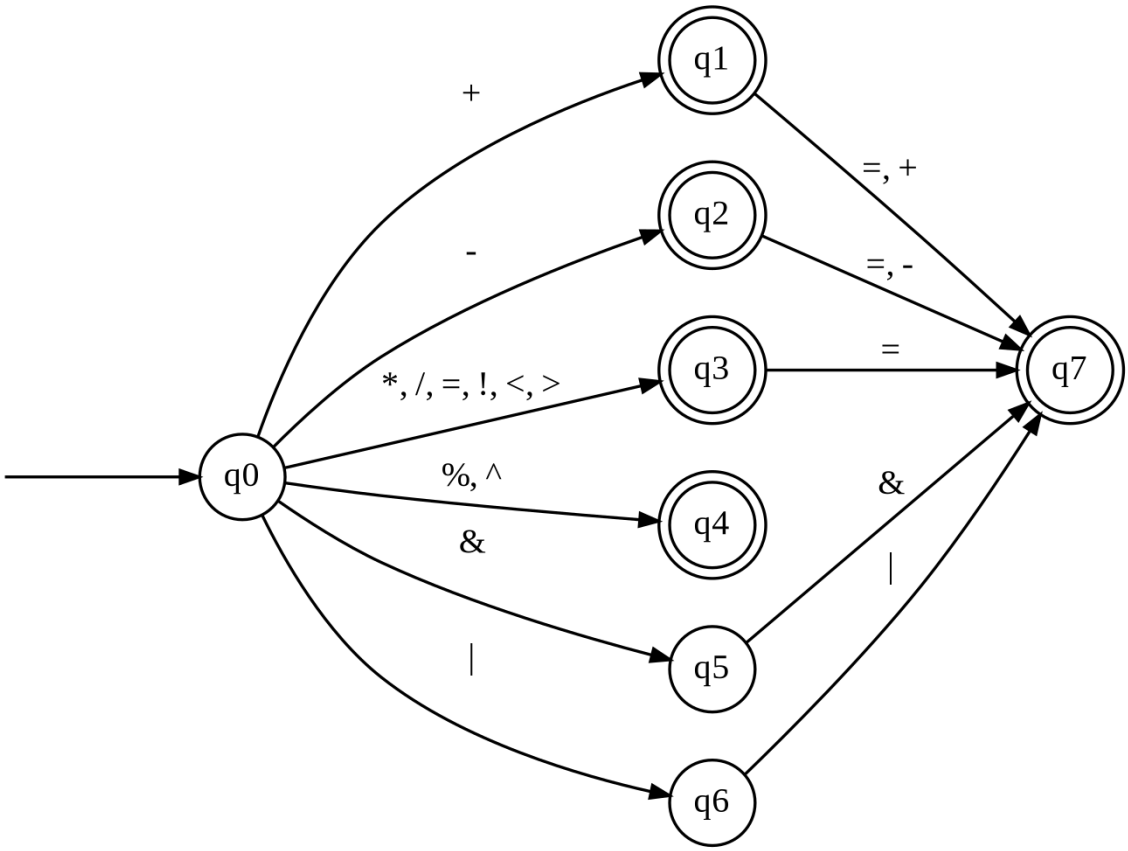
- Regular Expression: (==|!=|<=|>=|<|>)

4.6. Update Operators

Operator	Example	Description
++	`++X or X++`	Increases the value of operand X by 1. The operand must be a plain number. For a value with a unit, write `X += 1m;` instead.
--	`--X or X--`	Decreases the value of operand X by 1. The operand must be a plain number.

- Regular Expression: (\+\+|--)

Operation-Symbol Machine:



5. Keywords and Reserved Words

5.1. Keywords

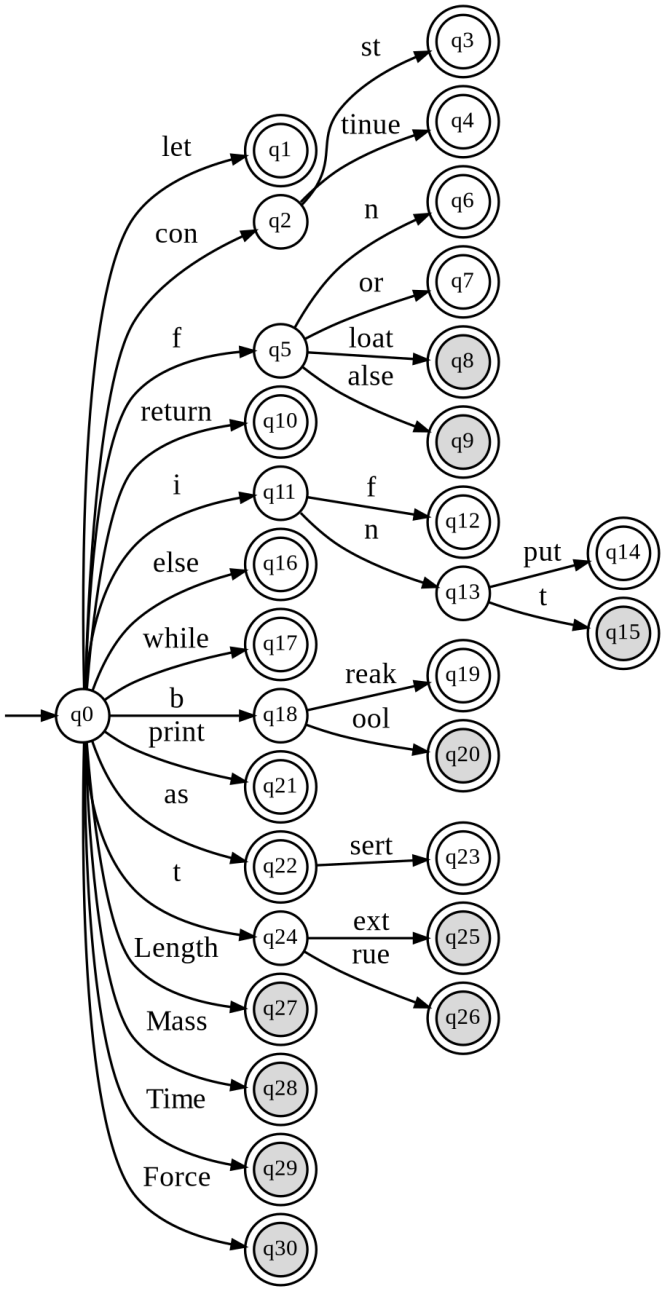
Keyword	Purpose
`let`	Declares a variable binding.
`const`	Declares a binding that cannot be reassigned.
`fn`	Begins a function declaration.
`return`	Returns a value from a function.
`if`	Begins a conditional statement.

`else`	Introduces the alternative branch of a conditional.
`for`	Begins a counted or clause-controlled loop.
`while`	Begins a condition-controlled loop.
`break`	Exits the nearest enclosing loop.
`continue`	Skips to the next iteration of the nearest enclosing loop.
`print`	Writes one or more expressions to the output.
`input`	Reads an input value.
`assert`	Requires a Boolean condition to hold.
`as`	Forms an explicit postfix unit conversion.

5.2. Reserved Words

Category	Reserved words	Purpose
Primitive types	`int, float, bool, text`	Name the four primitive declaration types.
Boolean literals	`true, false`	Represent the two Boolean values.
Named dimensions	`Length, Mass, Time, Force`	Constrain declarations and function contracts by physical dimension.

Keyword and Reserved-Word Machine:

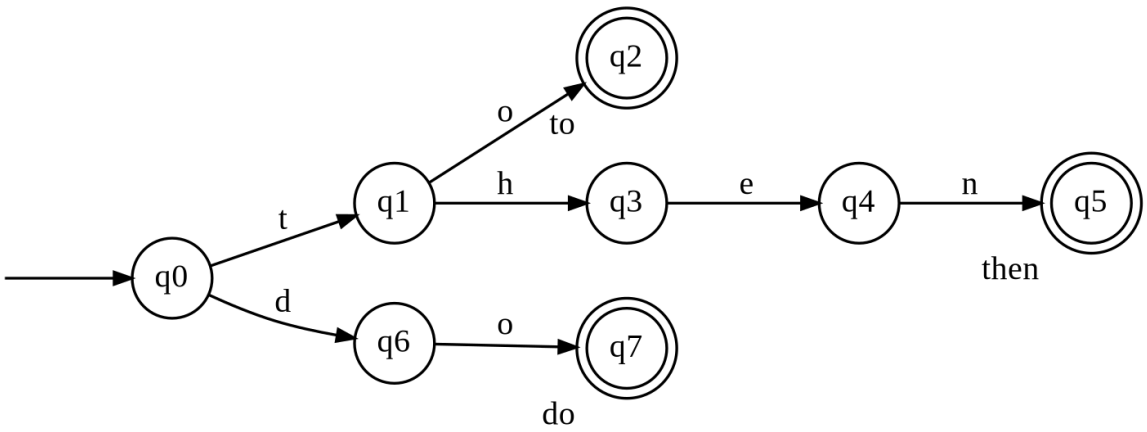


Double circle = keyword Shaded double circle = reserved word
A multi-letter edge stands for a chain of single-letter transitions.

6. Noise Words

Noise word	Used with	Form without the noise word	Equivalent form with the noise word
`to`	`as` conversion	`distance as km`	`distance as to km`
`then`	`if` conditional	`if (condition) { ... }`	`if (condition) then { ... }`
`do`	`while` loop	`while (condition) { ... }`	`while (condition) do { ... }`
`do`	`for` loop	`for (...) { ... }`	`for (...) do { ... }`

Machine for Noise Words:

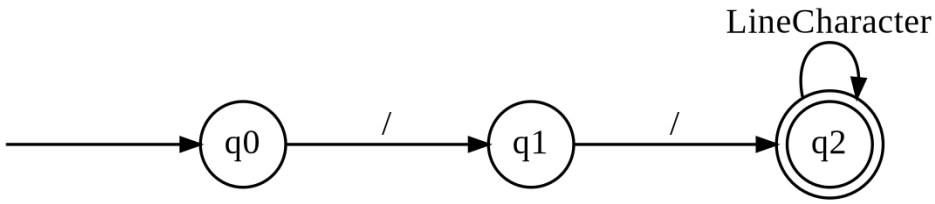


7. Comments

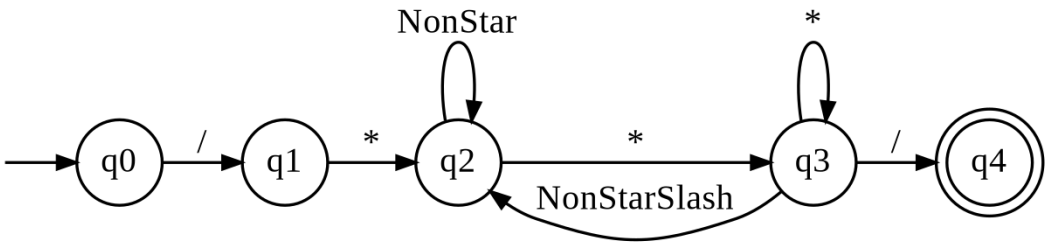
- Slash = { / }
- Star = { * }
- LineCharacter = Character – { <carriage-return>, <line-feed> }
- NonStar = Character – { * }
- NonStarSlash = Character – { *, / }
- SingleLineComment = Slash Slash LineCharacter*
- BlockComment = Slash Star (NonStar | Star Star* NonStarSlash)* Star Star* Slash

- Single-line comments are denoted by two forward-slashes `//`
- Block comments are denoted by the forward-slash and star pairs `/* */`

Machine for single-line comments:



Machine for multiple-line comments:



8. Blanks

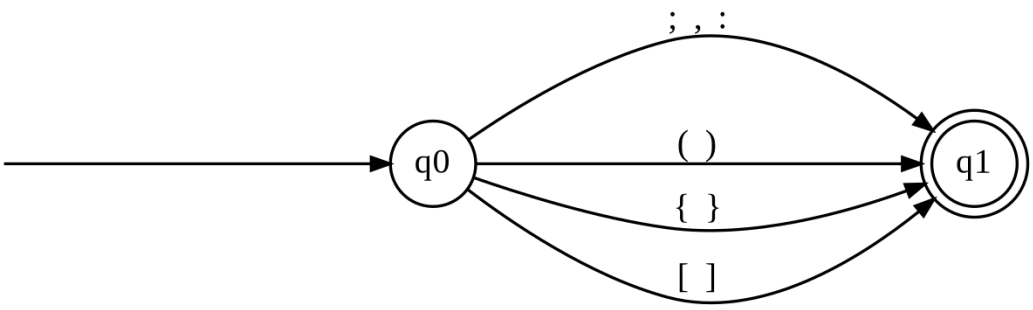
- SML recognizes spaces, horizontal tabs, carriage returns, and line feeds as whitespace.
- Whitespace separates tokens when they would otherwise run together, as in `let length`.
- Whitespace is optional around operators and delimiters. For example, `a+b` and `a + b` are equivalent.
- Whitespace cannot appear inside a multi-character operator. Thus, `+=` is one operator, while `+ =` is two separate symbols.
- A quantity's unit must be attached directly to its magnitude. Therefore, `100m` and `9.8[m/s^2]` are valid, while `100 m` and `9.8 [m/s^2]` are invalid.
- Line breaks do not terminate statements; semicolons do. Multiple statements may therefore appear on one line.
- A brace-delimited block may be written on one line or across several lines.
- Whitespace inside a string literal is part of the string and is not ignored.

9. Delimiters and Brackets

- Delimiter = { ;, (,), {, }, [,], ,, : }

Delimiter	Description
Semicolon (`;`)	Terminates a simple statement and separates clauses in a for header. It terminates statements rather than physical lines.
Parentheses (`(` and `)`)	group expressions and enclose conditions, function parameters, function arguments, and for headers.
Curly braces (`{` and `}`)	enclose function bodies and all conditional or iterative statement bodies, including a body containing only one statement.
Square brackets (`[` and `]`)	enclose compound units in quantity literals, exact-unit declaration annotations, and compound conversion targets. For example, `9.8[m/s^2]` and `let force: [N] = 12kN;` use square brackets for unit syntax.
Comma (`,`)	separates function parameters, function arguments, and expressions passed to print.
Colon (`:`)	introduces an annotation. It separates a declared name from its annotation, as in let span: Length = 5m;, and introduces a function's return annotation after its parameter list.

Machine for Delimiters:



10. Free-and-fixed-field formats

SML uses a free-field format. Source elements are not required to begin in particular columns, and indentation does not determine program structure.

- Spaces, tabs, and line breaks may be used freely between tokens where separation is permitted.
- Statements may span multiple lines, and multiple statements may appear on one line.
- A semicolon terminates a simple statement; a physical line break does not.

- Curly braces determine the boundaries of functions, conditionals, and loops. Indentation is recommended for readability but has no syntactic meaning.
- Comments are treated like whitespace between tokens.
- Attached quantity syntax remains an exception: no whitespace or comment may separate a magnitude from its unit symbol or opening square bracket.
- Whitespace inside a string literal is preserved as part of the string.

11. Expression

11.1. Precedence and Associativity

The levels below are ordered from highest to lowest precedence.

Level	Form or operator	Associativity
1	literals, identifiers, calls, and parenthesized expressions	Primary
2	postfix conversion as [to] UnitTarget	Left-to-right postfix
3	power ^	Right-to-left
4	unary !, +, -	Right-to-left
5	*, /, %	Left-to-right
6	+, -	Left-to-right
7	<, <=, >, >=	Non-chaining
8	==, !=	Non-chaining
9	&&	Left-to-right
10	||	Left-to-right

11.2. Expression Syntax

The table summarizes the valid expression forms. Angle brackets identify a category that must be replaced by a valid SML form.

Expression form	Syntax	Example
Literal	`<literal>`	`9.5s`
Identifier	`<identifier>`	`distance`
Parenthesized expression	`(<expression>)`	`(distance / time)`

Function call	`<identifier>(<arguments>)`	`speed(distance, time)`
Input expression	`input()`	`input()`
Unit conversion	`<expression> as [to] <unit-target>`	`3mi as km`
Unary expression	`<unary-operator><expression>`	`-distance`
Power expression	`<expression> ^ <expression>`	`length ^ 2`
Multiplicative expression	`<expression> <operator> <expression>`	`length * length`
Additive expression		`5m + 3m`
Relational expression		`distance < limit`
Equality expression		`count == 5`
Logical expression		`valid && ready`

Prefix and postfix increment and decrement apply only to dimensionless numeric variables. A dimensioned variable must be changed with an explicit unit-bearing assignment, such as `length += 1m;`.

11.3. Expression Rules

Operation	Rule	Example
+ and -	Operands must have the same dimension and exact unit representation.	`5m + 2m`
%	Operands must have the same exact unit representation.	`5m % 2m`
*	Operand units are multiplied to derive the result unit.	`3m * 2m` produces `[m^2]`
/	Operand units are divided to derive the result unit.	`100m / 9.5s` produces `[m/s]`
^	A dimensioned base requires a compile-time integer exponent; a dimensionless base may use an integer or decimal exponent.	`3m ^ 2` produces `[m^2]`
<, <=, >, >=	Comparable operands produce a Boolean result and require an explicit common unit representation.	`5km < (3mi as km)`

== and !=	Comparable primitive values or quantities with the same exact unit representation produce a Boolean result.	`5m != 3m`
!, &&, ||	Operate on Boolean values; && and || use short-circuit evaluation.	`valid && safe`
as	Explicitly converts a quantity to a compatible unit target.	`3mi as km`

11.4. Unit Expressions

A UnitExpression describes a compound unit inside square brackets. It is used by compound quantity literals, exact-unit annotations, and compound conversion targets.

- SignedInteger = ["-"], Digits+ ;
- UnitPower = UnitSymbol, ["^", SignedInteger] ;
- UnitProduct = UnitPower, { "*", UnitPower } ;
- UnitDenominator = UnitPower | "(", UnitProduct, ")" ;
- UnitExpression = UnitProduct, ["/", UnitDenominator] ;

The following restrictions keep unit expressions manageable for lexical and syntactic analysis:

- Multiplication must be written explicitly with *.
- Only one top-level / is permitted; repeated or nested division is invalid.
- A denominator containing multiple factors must be enclosed in parentheses.
- A grouped unit expression cannot be raised to a power.
- Numeric scale factors are not allowed inside unit expressions.
- A unit expression may contain at most eight written unit-factor occurrences.
- Written unit exponents must range from -16 through 16.
- Normalized dimension exponents must remain from -32 through 32.
- The contents of one unit annotation may contain at most 128 ASCII characters, including whitespace.

Accepted unit expressions include:

```
9.8[m/s^2]
1[kg*m/s^2]
1[kg/(m*s^2)]
```

Rejected forms include:

```
1[kg/m/s^2]  // repeated division
1[m/(kg/s)]  // nested division
1[(m/s)^2]   // power applied to a group
1[N m]       // implicit multiplication
1[1000*m]    // numeric scale factor
```

12. Statements

12.1. Declaration Statements

Declaration syntax	Example
`let <identifier>;`	`let count;`
`let <identifier> = <expression>;`	`let count = 5;`
`let <identifier> = <quantity-expression>;`	`let span = 5m;`
`let <identifier>: <annotation>;`	`let distance: Length;`
`let <identifier>: <annotation> = <expression>;`	`let distance: m = 100m;`

Declaration form	Example
Inferred dimensionless value	`let count = 5;`
Primitive-type annotation	`let count: int = 5;`
Inferred quantity	`let span = 5m;`
Named-dimension annotation	`let span: Length = 5m;`
Exact-unit annotation	`let exactSpan: [m] = 5m;`
Constant binding	`const gravity: [m/s^2] = 9.8[m/s^2];`

Only one annotation category may be used. Combined forms such as `float[m]`, `Length(m)`, and `float Length` are invalid. Each declaration introduces one binding; comma-separated declarations are not permitted.

12.2. Input and Output Statements

Syntax	Example
`print(<expression>, ...);`	`print("Speed: ", speed);`
`let <identifier>: <annotation> = input();`	`let height: [m] = input();`
`<identifier> = input();`	`height = input();`

The `print` statement accepts one or more expressions. When a quantity is printed, its unit representation is included with its magnitude.

```
let speed = 100m / 9.5s;
print("Speed: ", speed); // “Speed: 10.5263 m/s”
```

** Output values in examples may be rounded for illustration.*

The input form `input()` is an expression used directly as the initializer or right-hand side of a binding whose expected primitive type or exact unit is known:

```
let height: [m] = input();
print("Height: ", height);

height = input();
```

An unannotated declaration such as `let value = input();` is invalid because SML cannot infer its expected type or unit from the input expression alone.

12.3. Assignment Statements

Syntax	Example	Meaning
`<identifier> = <expression>;`	`a = b;`	Stores the value of an expression.
`<identifier> += <expression>;`	`length += 1m;`	Adds an explicitly stated value.
`<identifier> -= <expression>;`	`length -= 1m;`	Subtracts an explicitly stated value.

`<identifier> (*= | /=) <dimensionless-expression>;`	`scale *= 2;`	Multiplies or divides without changing the binding's dimension.
`++<identifier>;` or `-- <identifier>;`	`++count;`	Prefix update of a dimensionless numeric variable.
`<identifier>++;` or `<identifier>--;`	`count++;`	Postfix update of a dimensionless numeric variable.

All of the following are valid assignment forms in SML:

```
let b = 5m;
let g = 2m;
let a = 0m;
let area: [m^2] = 0[m^2];

a = b;
a = b + 3m;
area = (b + 3m) * g / 24;
```

The final expression produces `area` rather than length because it multiplies two `Length` quantities. Assigning that expression to `a` would be invalid, while assigning it to `area` is valid.

Compound assignments and postfix updates are also supported:

```
a += 1m;
a *= 2;
a++;
```

12.4. Conditional Statements

Form	Syntax
`if`	`if (<expression>) [then] <block>`
`if-else`	`if (<expression>) [then] <block> else <block>`
`if-else if-else`	`if (<expression>) [then] <block> else if (<expression>) [then] <block> else <block>`

The condition must produce a Boolean value. Braces are required even when a branch contains only one statement.

- An `if` statement:

```
if (load > 100kN) {
    print("High load");
}
```

- An `if-else` statement:

```
if (load > 100kN) {
    print("High load");
} else {
    print("Normal load");
}
```

- An `if-else if-else` statement:

```
if (load > 100kN) {
    print("High load");
} else if (load == 100kN) {
    print("At the limit");
} else {
    print("Below the limit");
}
```

- A nested conditional:

```
if (load > 0kN) {
    if (load > 100kN) {
        print("Positive high load");
    } else {
        print("Positive normal load");
    }
} else {
    print("No positive load");
}
```

12.5. Iterative Statements

SML provides `while` and C-style `for` loops. Their conditions must produce Boolean values, and their bodies always use braces.

Form	Syntax
`while` loop	`while (<expression>) [do] <block>`
`for` loop	`for (<initialization>; <condition>; <update>) [do] <block>`

- A loop containing one statement still requires braces:

```
for (let i: int = 0; i < 3; i++) {
    print(i);
}
```

- A `while` loop:

```
let fuel: [kg] = 10kg;

while (fuel > 0kg) do {
    print("Fuel left: ", fuel);
    fuel -= 1kg;
}
```

- Nested for loops:

```
for (let row: int = 0; row < 3; row++) {
    for (let column: int = 0; column < 3; column++) {
        print(row, ",", column);
    }
}
```

12.7. Function Declarations and Program Entry

A function declaration begins with fn, followed by an identifier, a parameter list, an optional return annotation, and a brace-delimited body. Parameters and return values may use primitive, dimensional, or exact-unit annotations. A function without a return annotation returns no value.

Form	Syntax
Function without parameters	`fn <identifier>() [: <return-annotation>] <block>`
Function with parameters	`fn <identifier>(<parameters>) [: <return-annotation>] <block>`
Parameter	`<identifier>: <annotation>`
Return statement	`return [<expression>];`

For example:

```
fn speed(distance: Length, time: Time): [m/s] {
    return (distance as m) / (time as s);
}
```

A parameterless function named main serves as the conventional entry point of an SML program. The spelling main is an ordinary case-sensitive identifier rather than a keyword or reserved word.

```
fn main() {
    let result = speed(100m, 10s);
    print("Speed: ", result);
}
```